

X-KANformer

Haiyang Gao
Imperial College London
London, United Kingdom
haiyang.gao26@imperial.ac.uk

Shivam Sharma
Yale University
New Haven, USA
shivam.sharma@yale.edu

Abstract

Time-series forecasting is central to financial decision-making. Financial data are noisy, non-stationary, and often heavy-tailed, which makes modelling extremely challenging with conventional parametric methods. Most current state of the art time-series forecasting models are Transformer-based architectures that improve long-range dependency modeling through attention mechanism followed by a standard highly over-parametrized feed-forward neural network (MLP) as the nonlinear transformation module, which severely hinders interpretability (and may as well accuracy). In this paper, we introduce a novel time-series forecasting model: X-KANformer which blends attention mechanism with Kolmogorov-Arnold Networks (KANs) thereby providing flexible edge-wise nonlinear transformations and improving out-of-sample portfolio performance in our experiments. Additionally, KANs have shown promising results in achieving better scaling laws than MLPs, which is a highly sought-after model virtue, especially for the Hedge fund industry. X-KANformer combines endogenous stock-return time-series with exogenous market information (such as market sentiment, etc.) through self-attention and cross-attention, and then maps the learned (contextualized) representations directly into portfolio weights that would maximize the out-of-sample Sharpe-ratio. Our model further employs Selective Learning during training to reduce the influence of noisy, anomalous, and less generalizable financial observations. Empirically, our ablation study strongly suggests that KAN-enhanced Transformer architectures offer a promising direction for integrating long-range temporal modeling, nonlinear function approximation, and Sharpe-ratio-driven portfolio allocation.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Applied computing** → *Economics*.

Keywords

financial time-series forecasting, portfolio optimization, Transformer, Kolmogorov–Arnold networks, selective learning

ACM Reference Format:

Haiyang Gao and Shivam Sharma. 2026. X-KANformer. In *Proceedings of The 7th ACM International Conference on AI in Finance (ICAIF '26)*. ACM, New York, NY, USA, 9 pages.

1 Introduction

Financial time series forecasting has long been a central problem in empirical finance, as accurate forecasts of asset returns, volatility, and risk dynamics are directly related to portfolio allocation,

risk management, derivative pricing, and trading strategy design. Unlike many physical time series, financial data are typically noisy, non-stationary, heavy-tailed, and characterized by volatility clustering, making prediction particularly challenging. Classical statistical models laid the foundation for this field. For example, autoregressive moving average (ARMA) models provide a systematic framework for modeling linear temporal dependence [3], while the autoregressive conditional heteroskedasticity (ARCH) model of [7] and its generalized extension, the GARCH model of [2], became standard tools for capturing time-varying volatility in financial markets. These models have been widely used because they offer interpretable parametric descriptions of conditional mean and conditional variance dynamics. However, their reliance on relatively restrictive assumptions and limited capacity to capture complex nonlinear dependencies motivates the development of more flexible forecasting models, in particular to capture high-dimensional phenomenon in stock markets where cross-sectional interactions and long-range temporal patterns are essential for portfolio decision-making.

In recent years, the rapid development of deep learning has shifted financial time series forecasting from purely parametric parsimonious models towards highly expressive neural architectures. Among these methods, Transformer-based models have become one of the dominant research directions for long-sequence time series forecasting, primarily because their self-attention mechanism [31] is well suited to modeling long-range temporal dependencies. Starting from the original Transformer architecture [31], a series of forecasting-specific variants have been proposed to address the computational and statistical challenges of long sequences. Informer improves efficiency through ProbSparse self-attention [36]; Autoformer incorporates decomposition and autocorrelation mechanisms to better capture trend and seasonal structures [33]; FEDformer further combines seasonal-trend decomposition with frequency-domain representations [37]; and Non-stationary Transformer explicitly tackles distribution shifts in real-world non-stationary series [23]. More recent models continue this line of development: PatchTST uses subseries-level patching and channel independence to improve long-term forecasting [26]; iTransformer reconsiders the role of attention by applying it across variate tokens rather than temporal tokens [22]; Fredformer addresses frequency bias in Transformer forecasting [27]; DeformableTST reduces the over-reliance on patching through deformable sparse attention [25]; and TimeXer extends Transformer forecasting to settings with exogenous variables by jointly modeling endogenous temporal patches and exogenous variate-level information [32]. This trend is especially relevant for financial forecasting, where asset returns are influenced by long-memory effects, cross-asset interactions, macroeconomic conditions, and abrupt regime changes.

The effectiveness of Transformer-based models mainly comes from two architectural ingredients. First, the attention block learns data-dependent weights over different time steps or variables, allowing the model to selectively focus on historically relevant observations and thereby capture long-range temporal dependencies that are difficult for recurrent [15] or local models to preserve. Second, the attention block is followed by a deep feed-forward neural network, which applies nonlinear transformations to the learned representations. This component increases the expressive power of the model and enables it to capture complex nonlinear relationships among market signals, volatility patterns, and asset-level interactions. These properties make Transformer-based architectures a natural candidate for portfolio-oriented forecasting tasks, where the objective is not only to predict future price movements but also to extract predictive representations that support allocation decisions and risk-adjusted performance optimization.

While most Transformer based forecasters retain the standard feed-forward neural network (also called multi-layer perceptron: MLP) as the nonlinear transformation module after attention, this design choice is rarely examined in isolation, especially in financial applications. An emerging alternative is the KANs, which is motivated by the Kolmogorov-Arnold representation theorem and replaces scalar weights or fixed node-wise activations with learnable univariate functions, commonly parameterized by splines [24]. This structure is attractive in Finance because it preserves the ability to model nonlinear relationships while potentially using a more compact (parsimonious) and hence interpretable functional representation than conventional MLPs. Recent studies have begun to explore KANs in long-term time series forecasting, suggesting that adaptive basis functions can indeed serve as a flexible modeling core for capturing complex temporal patterns [17]. Nevertheless, their role in financial time series forecasting remains largely underexplored, with only preliminary evidence on standard KANs for stock price prediction and limited understanding of how KANs-based nonlinear modules interact with attention-based architectures [28]. In this paper, we carry out a comprehensive and detailed analysis of effectiveness of KANs in financial time series forecasting by carefully replacing the neural network component (MLPs) in the Transformer with KANs, and investigating its effect on portfolio-oriented prediction and Sharpe-ratio-driven allocation.

This leads to the main contribution of our study. We develop a portfolio-oriented Attention-KANs based model: **X-KANformer** that directly maps predictive information into portfolio weights. Specifically, the model takes as input a panel of historical stock returns, Fama-French three-factor variables, and a market news sentiment signal, with a total input dimension of 34 [Sections 4.1 and 4.2]. The output is a vector of portfolio weights optimized toward maximizing the Sharpe ratio, a standard risk-adjusted performance criterion in portfolio construction [29]. This formulation is important because, in investment applications, accurate return prediction is not sufficient by itself: the ultimate objective is to translate predictive signals into allocations that deliver superior compensation per unit of risk.

More precisely, our model has three key distinctive features: First, we fuse Transformer-based temporal modeling (the attention block) with KANs by carefully replacing the neural network component in the Transformer with KANs-style nonlinear transformations. To

the best of our knowledge, this design remains underexplored in financial time series forecasting and provides a direct way to examine whether learnable spline-based nonlinearities can improve portfolio prediction. We note that (along with improved interpretability and prediction accuracy) one of the central motivations for using KANs in financial time series context is to exploit *Virtue of Complexity* (VoC) phenomenon [18–20]; “KANs can beat the curse of dimensionality when there is a compositional structure in data, achieving better scaling laws than MLPs” [24]. Second, in addition to historical returns, the model’s novel design [Section 3] incorporates cross-sectional and market-level information, including the Fama-French three factors [8] and market news sentiment [Section 4.1], allowing the model to exploit both time-series dynamics and broader systematic signals. Third, we train the model using Adam optimizer [21] together with selective learning [12], which masks uncertain and anomalous timesteps during optimization so that the model focuses on more generalizable patterns.

We perform a thorough and in-depth empirical investigation: our ablation study [Section 4.5.1] strongly suggests that KAN-enhanced Transformer architectures offer a promising direction for integrating long-range temporal modeling, nonlinear function approximation, and Sharpe-ratio-driven portfolio allocation. We also note that the resulting strategy achieves an out-of-sample Sharpe ratio of 1.375. (For context, the average Sharpe ratio on the JKP Common Task Framework benchmark leaderboard at the time of evaluation is 1.321.)

1.1 Related work/Literature review

Recent research has increasingly applied machine learning and deep learning to asset pricing, return prediction, and portfolio optimization. In empirical asset pricing, Gu et al. [13] show that machine learning methods can generate substantial economic gains by capturing nonlinear interactions among firm characteristics, while Chen et al. [4] use deep neural networks to estimate flexible asset pricing models with rich conditioning information and time-varying economic states. A closely related line of work studies end-to-end portfolio optimization: Zhang et al. [35] directly optimize portfolio weights toward the Sharpe ratio, and Zhang et al. [34] develop a general deep learning framework that bypasses separate return and covariance estimation. In parallel, recent time series forecasting studies have developed Transformer-based architectures for long-horizon prediction, such as Informer, FEDformer, PatchTST, and TimeXer [26, 32, 36, 37], while newer work explores KANs as an alternative nonlinear modeling component [17, 24].

1.2 Organization

The remainder of this paper is organized as follows. Section 2 formulates the portfolio-oriented forecasting problem and explains the motivation for directly learning portfolio weights under a Sharpe-ratio objective. Section 3 presents the X-KANformer architecture. Section 4 describes the data sources, rolling-window construction, loss function, and training protocol, and then reports the leaderboard comparison and ablation results. Finally, Section 5 concludes the paper, discusses the limitations of the current analysis, and outlines directions for future research.

2 Preliminary

This study considers a portfolio-oriented forecasting problem. The goal is to learn an allocation rule $f_\theta(\cdot)$ that directly converts historical financial information into portfolio weights that would maximize the Sharpe ratio. In this sense, the forecasting output is a portfolio decision, and the quality of the forecast is judged by the realized performance of that decision in the next period. We next outline the forecasting problem.

At each decision date t , X-KANformer takes as input a historical window X_t containing endogenous stock-return information and exogenous market-level signals. Based on this information, the model produces the portfolio weights $\hat{\mathbf{w}}_{t+1}$ for the next investment period: $\hat{\mathbf{w}}_{t+1} = f_\theta(X_t)$. After the next-period returns $\mathbf{r}_{t+1}$ are realized, the portfolio return $r_{p,t+1}$ is computed as $r_{p,t+1} = \hat{\mathbf{w}}_{t+1}^\top \mathbf{r}_{t+1}$. Thus, the model is evaluated through the return generated by its predicted weights, rather than through the pointwise accuracy of return forecasts.

This formulation is motivated by the mismatch between return prediction and portfolio decision-making. If a model first predicts future returns, a separate trading or market-timing rule is still required to translate those forecasts into portfolio weights. The final investment outcome then depends not only on the accuracy of the return forecasts, but also on the additional allocation rule, leverage constraint, and rebalancing decision. In noisy financial markets, this two-stage procedure may amplify prediction errors and produce unstable portfolio weights. To reduce this mismatch, we formulate the problem as a direct portfolio-weight learning task, where the model is optimized toward the Sharpe ratio of the

realized portfolio returns: $\text{SR} = \frac{\mathbb{E}[r_{p,t+1}]}{\sqrt{\text{Var}(r_{p,t+1})}}$. Equivalently, the

desired allocation rule can be viewed as the one that maximizes the Sharpe ratio induced by the model-generated portfolio returns: $\theta^* = \arg \max_{\theta} \frac{\mathbb{E}[f_\theta(X_t)^\top \mathbf{r}_{t+1}]}{\sqrt{\text{Var}(f_\theta(X_t)^\top \mathbf{r}_{t+1})}}$. This setting aligns the forecasting task with the economic objective of portfolio management: the model should not merely describe future returns but should produce allocations that achieve stronger compensation per unit of risk.

3 The Model and Methodology

3.1 X-KANformer: Structure Overview

X-KANformer is an encoder-only portfolio-allocation model designed for financial time series with exogenous variables. The model takes a historical input window and maps it directly into portfolio weights that would maximize the Sharpe ratio. Endogenous information is represented at the patch level, exogenous information is represented at the variate level, and an endogenous global token bridges the two information sources through cross-attention. In addition, the conventional Transformer feed-forward network is replaced by KANs, which provide learnable edge-wise nonlinear functions, Figure 1.

Given an input window (look-back window) of length L , the model input is divided into endogenous and exogenous components. Let $\mathbf{X}^{\text{en}} = [\mathbf{x}_1^{\text{en}}, \dots, \mathbf{x}_L^{\text{en}}] \in \mathbb{R}^{C_y \times L}$, denote the endogenous input. The exogenous input is denoted by

$$\mathbf{X}^{\text{ex}} = [\mathbf{x}_1^{\text{ex}}, \dots, \mathbf{x}_L^{\text{ex}}] \in \mathbb{R}^{C_e \times L}. \quad (1)$$

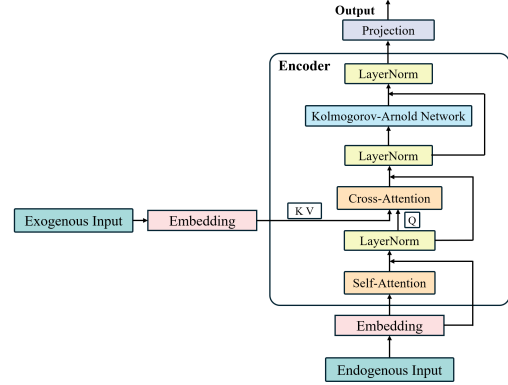


Figure 1: Architecture of the X-KANformer.

Equivalently, the raw input can be written as $\mathbf{X} = [\mathbf{X}^{\text{en}}, \mathbf{X}^{\text{ex}}] \in \mathbb{R}^{(C_y + C_e) \times L}$.

The model produces a portfolio weight vector $\mathbf{w} = [w_1, \dots, w_{C_y}]^\top \in \mathbb{R}^{C_y}$, where w_i is the allocation assigned to asset i .

From a portfolio-management perspective, X-KANformer can be viewed as a state-dependent allocation rule. The endogenous return histories describe asset-specific market dynamics, whereas the exogenous variables summarize common market conditions that may affect all assets simultaneously. The attention modules determine which parts of the available information are relevant at the current decision date, the KAN module transforms these contextualized signals through flexible nonlinear functions, and the projection head converts the resulting representation into a feasible portfolio, Figure 1. Hence, the architecture is not designed merely to forecast each asset return separately; it learns how historical information should be translated into relative portfolio allocations.

3.2 Input Embedding

3.2.1 Patch Embedding for Endogenous Variables. Let P denote the patch length and $M = L/P$ the number of non-overlapping patches. For the i -th endogenous variable, the historical sequence is divided into $\mathbf{p}_{i,m} = [x_{i,(m-1)P+1}, \dots, x_{i,mP}]^\top \in \mathbb{R}^P$, $m = 1, \dots, M$. Each patch is projected into the d -dimensional latent space and combined with a fixed sinusoidal positional encoding: $\mathbf{e}_{i,m}^{\text{en}} = \mathbf{W}_p \mathbf{p}_{i,m} + \mathbf{r}_m$, where $\mathbf{W}_p \in \mathbb{R}^{d \times P}$ and $\mathbf{r}_m \in \mathbb{R}^d$ denotes the positional encoding of the m -th patch. We use the standard absolute sinusoidal positional encoding introduced by Vaswani et al. [31].

A learnable global token $\mathbf{g}_i \in \mathbb{R}^d$ is appended to the patch tokens: $\mathbf{E}_i^{\text{en}} = [\mathbf{e}_{i,1}^{\text{en}}, \dots, \mathbf{e}_{i,M}^{\text{en}}, \mathbf{g}_i] \in \mathbb{R}^{(M+1) \times d}$. Stacking all endogenous variables gives $\mathbf{E}^{\text{en}} = [\mathbf{E}_1^{\text{en}}, \dots, \mathbf{E}_{C_y}^{\text{en}}]^\top \in \mathbb{R}^{C_y \times [(M+1) \times d]}$. This representation allows self-attention to capture temporal dependencies among patches of each endogenous variable, while the global token summarizes the variable-level state.

For an individual asset, each patch represents a local configuration of returns over a relatively short interval rather than a single isolated observation. This representation is economically useful because investment signals often depend on patterns formed over multiple periods, such as persistent price movements, short-term

reversals, drawdowns, and changes in return variability. By embedding an entire patch, the model is encouraged to extract such local configurations while reducing its sensitivity to any single noisy return observation. The global token can then be interpreted as a learned summary of the asset's state over the complete look-back window, potentially combining information about recent performance, risk, and the relative importance of different historical episodes.

3.2.2 Inverted Embedding for Exogenous Variables. The exogenous input is represented using an inverted embedding. Recall from (1) that $\mathbf{X}^{\text{ex}} = [\mathbf{x}_1^{\text{ex}}, \dots, \mathbf{x}_L^{\text{ex}}] \in \mathbb{R}^{C_e \times L}$, where each column $\mathbf{x}_t^{\text{ex}} \in \mathbb{R}^{C_e}$ contains the exogenous observations at time step t .

Unlike the endogenous embedding, which divides each endogenous sequence into temporal patches, the inverted embedding treats the complete historical trajectory of each exogenous variable as one token. Specifically, for the j -th exogenous variable, define its length- L trajectory as $\mathbf{x}_j^{\text{ex}} = [x_{j,1}^{\text{ex}}, \dots, x_{j,L}^{\text{ex}}]^\top \in \mathbb{R}^L$, $j = 1, \dots, C_e$. This trajectory is projected into the d -dimensional latent space through a shared linear embedding: $\mathbf{e}_j^{\text{ex}} = \text{Dropout}(\mathbf{W}_e^\top \mathbf{x}_j^{\text{ex}} + \mathbf{b}_e) \in \mathbb{R}^d$, where $\mathbf{W}_e \in \mathbb{R}^{L \times d}$, $\mathbf{b}_e \in \mathbb{R}^d$. Here, $\mathbf{W}_e$ is a learnable projection matrix that maps the complete length- L trajectory of each exogenous variable into the d -dimensional latent space, while $\mathbf{b}_e$ is the corresponding learnable bias vector. The dropout operation is applied following [30]. Applying the same projection to all C_e exogenous variables gives $\mathbf{E}^{\text{ex}} = \text{Dropout}(\mathbf{X}^{\text{ex}} \mathbf{W}_e + \mathbf{b}_e) = [\mathbf{e}_1^{\text{ex}}, \dots, \mathbf{e}_{C_e}^{\text{ex}}]^\top \in \mathbb{R}^{C_e \times d}$.

Therefore, each row of $\mathbf{E}^{\text{ex}}$ is a variate-level token that encodes the complete history of one exogenous variable over the input window. The resulting exogenous tokens are subsequently used as the keys and values in the cross-attention module.

The exogenous variables are not assets to which the model assigns portfolio weights. Instead, they serve as common state variables that condition the allocation decision. In the present application, the market excess return, SMB, and HML summarize broad market, size, and value conditions, while the news-sentiment series provides information about the prevailing macroeconomic information environment. Representing the complete trajectory of each exogenous variable as one token allows the model to encode not only its latest observation but also how the corresponding market condition has evolved throughout the look-back window.

This representation is particularly appropriate for portfolio allocation because the same market state may have different implications for different assets. The exogenous tokens therefore provide common conditioning information, while the subsequent cross-attention mechanism allows each asset representation to respond differently to that information.

3.3 Encoder Layer

The model stacks N_{enc} encoder layers. The input of the ℓ -th encoder layer is $\mathbf{H}^{(\ell-1)} \in \mathbb{R}^{C_y \times [(M+1) \times d]}$, where $\mathbf{H}^{(0)} = \mathbf{E}^{\text{en}}$. The exogenous embedding $\mathbf{E}^{\text{ex}}$ is shared across all encoder layers as the source of key and value tokens for cross-attention. Each encoder layer consists of endogenous self-attention, exogenous cross-attention, and a KAN-based feed-forward mapping.

3.3.1 Endogenous Self-Attention. For each endogenous variable, the layer first applies multi-head self-attention [31] to its patch tokens and global token, followed by a residual connection [14], dropout [30], and layer normalization [1]:

$$\tilde{\mathbf{H}}^{(\ell)} = \text{LayerNorm}\left(\mathbf{H}^{(\ell-1)} + \text{Dropout}\left(\text{Attention}(\mathbf{H}^{(\ell-1)}, \mathbf{H}^{(\ell-1)}, \mathbf{H}^{(\ell-1)})\right)\right).$$

The attention mechanism is the standard scaled dot-product attention: $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}}\right)\mathbf{V}$, where d_h denotes the dimensionality of each attention head.

Within each endogenous channel, self-attention compares the asset's current representation with its representations from different historical patches. Economically, this allows the model to assign different relevance to different past market episodes. For example, an earlier patch associated with a persistent trend, a reversal, or an elevated-risk period may receive a larger attention weight when it is informative for the current allocation decision. Unlike a fixed-lag autoregressive specification, these relevance weights are data dependent and can vary across assets and decision dates.

3.3.2 Exogenous Cross-Attention. Following the endogenous-exogenous interaction mechanism introduced in TimeXer [32], the output of endogenous self-attention is decomposed into local patch tokens and global tokens: $\tilde{\mathbf{H}}^{(\ell)} = [\tilde{\mathbf{P}}^{(\ell)}, \tilde{\mathbf{G}}^{(\ell)}]$, where $\tilde{\mathbf{P}}^{(\ell)} \in \mathbb{R}^{C_y \times M \times d}$, $\tilde{\mathbf{G}}^{(\ell)} \in \mathbb{R}^{C_y \times d}$. Here, $\tilde{\mathbf{P}}^{(\ell)}$ contains the self-attended local patch representations, while $\tilde{\mathbf{G}}^{(\ell)}$ summarizes the endogenous sequences at the global-token level.

For cross-attention¹, the endogenous global tokens are used as queries, and the exogenous embedding provides the keys and values: $\tilde{\mathbf{G}}^{(\ell)} = \text{LayerNorm}\left(\tilde{\mathbf{G}}^{(\ell)} + \text{Dropout}\left(\text{MHCA}\left(\tilde{\mathbf{G}}^{(\ell)}, \mathbf{E}^{\text{ex}}, \mathbf{E}^{\text{ex}}\right)\right)\right)$. After cross-attention, the updated global tokens are concatenated with the local patch tokens: $\tilde{\mathbf{H}}^{(\ell)} = [\tilde{\mathbf{P}}^{(\ell)}, \tilde{\mathbf{G}}^{(\ell)}]$.

The cross-attention block can be understood as a state-conditioning mechanism. Each asset-level global token queries the same set of market-level variables, but the resulting attention pattern may differ across assets. Consequently, the model can learn that an asset's allocation signal should depend more strongly on a particular factor or on news sentiment under some market conditions, while relying more heavily on its own return history under others. This mechanism is related in spirit to state-dependent factor exposure, but it is more flexible than a conventional linear factor model with fixed coefficients. The model does not impose a constant linear beta between an asset and each exogenous variable. Instead, it learns a nonlinear and time-varying statistical dependence between the asset representation and the market-state representation.

3.4 KAN-Based Feed-Forward Mapping

A conventional Transformer encoder uses a two-layer feed-forward network of the form $\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x})$. In this model, the FFN is replaced by a KAN module [24]: $\mathbf{H}^{(\ell)} = \text{LayerNorm}\left(\tilde{\mathbf{H}}^{(\ell)} + \text{Dropout}\left(\text{KAN}\left(\tilde{\mathbf{H}}^{(\ell)}\right)\right)\right)$.

¹MHCA stands for Multi Head Cross Attention.

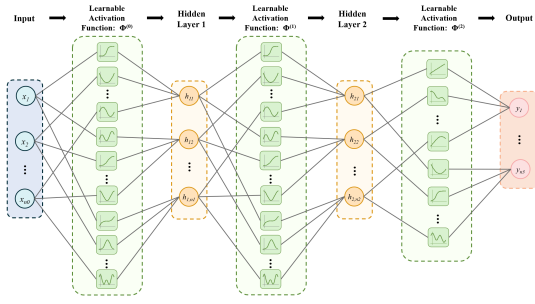


Figure 2: Illustration of KAN with learnable edge-wise activation functions. Unlike an MLP, which applies fixed activation functions at nodes, KAN learns univariate functions on edges and composes them across layers to construct high-dimensional nonlinear mappings.

For a KAN layer with input $\mathbf{u} \in \mathbb{R}^m$ and output $\mathbf{o} = \Phi(\mathbf{u}) \in \mathbb{R}^n$, the j -th coordinate of the output $\mathbf{o}$ can be written as $o_j = [\Phi(\mathbf{u})]_j = \sum_{i=1}^m \phi_{ji}(u_i)$, $j = 1, \dots, n$. Here, Φ denotes the vector-valued mapping implemented by the entire KAN layer, while ϕ_{ji} denotes the learnable univariate function associated with the edge from the i -th input to the j -th output. Thus, each output component o_j is obtained by summing the edge-wise transformations $\phi_{ji}(u_i)$ over all input dimensions.

Each edge function $\phi_{ji}(\cdot)$ consists of a base activation branch and a B-spline branch: $\phi_{ji}(u_i) = w_{ji}^{\text{base}} \text{SiLU}(u_i) + s_{ji} \sum_{r=1}^{G+k} w_{jir}^{\text{spline}} B_{ir}^{(k)}(u_i)$. The SiLU activation function [6] is defined as $\text{SiLU}(u_i) = u_i \sigma(u_i) = \frac{u_i}{1 + \exp(-u_i)}$. Here, $B_{ir}^{(k)}(\cdot)$ denotes the r -th B-spline basis function of order k constructed over a grid of size G for input coordinate i , and s_{ji} is a learnable spline scaling factor. The $G + k$ basis functions form the spline expansion of the edge function.

The learnable parameters associated with the edge from input coordinate i to output coordinate j are $\Theta_{ji} = \{w_{ji}^{\text{base}}, s_{ji}, w_{ji1}^{\text{spline}}, \dots, w_{ji(G+k)}^{\text{spline}}\}$.

The KAN used in the encoder can be written as $\text{KAN}(\mathbf{x}) = \Phi^{(2)}(\Phi^{(1)}(\Phi^{(0)}(\mathbf{x})))$. This design gives the encoder a high-capacity nonlinear mapping after attention-based contextualization. For a more detailed and thorough understanding of KANs, we refer the interested reader to [24].

3.5 Projection Head and Portfolio Weight Normalization

After the final encoder layer, the model obtains a latent representation that contains information extracted from the endogenous input, the exogenous variables, and the KAN-enhanced nonlinear transformation. Let this final representation be denoted by $\mathbf{z}^{\text{out}} = \text{Flatten}(\mathbf{H}^{(N_{\text{enc}})}) \in \mathbb{R}^D$, where D is the flattened hidden dimension after the encoder. The projection head maps this representation into asset allocation scores: $\mathbf{s} = \mathbf{W}_o \mathbf{z}^{\text{out}} + \mathbf{b}_o$, $\mathbf{s} \in \mathbb{R}^{C_y}$. The score vector is then transformed into a portfolio weight vector.

Under the long-only fully invested constraint, this normalization is implemented by the softmax function:

$$w_i = \frac{\exp(s_i)}{\sum_{j=1}^{C_y} \exp(s_j)}, \quad i = 1, \dots, C_y. \quad (2)$$

Therefore, $w_i \geq 0$, $\sum_{i=1}^{C_y} w_i = 1$. The final model output is $\mathbf{w} = [w_1, w_2, \dots, w_{C_y}]^T \in \mathbb{R}^{C_y}$.

The role of the KAN module is to translate the information selected by the attention blocks into nonlinear, allocation-relevant features. Financial relationships are frequently asymmetric and state dependent. For example, a large negative sentiment shock may have a different effect from a positive shock of the same magnitude, and the relevance of a factor realization may change between low-volatility and high-volatility periods. A fixed linear mapping cannot represent such threshold effects or changing marginal responses without manually introducing interaction terms. The learnable univariate edge functions in KAN provide a flexible way to represent these smooth nonlinear responses. Because the KAN is applied after self-attention and cross-attention, its inputs already contain information about both asset-specific return dynamics and the prevailing market state. The module can therefore learn how the marginal contribution of a latent signal changes with its magnitude and context.

4 Data and Experiments

4.1 The Data

The empirical analysis uses three classes of data.

First, stock return data are obtained from JKP Factors. The individual stock returns are drawn from the JKP Factors platform, which provides the global stock and factor database constructed by [16]. The data system covers the U.S. stock market and is updated regularly. It includes stock returns, stock characteristics, and factor returns. This thesis uses the stock return component and performs additional screening on the raw sample. Specifically, after handling missing values, rows and columns with more than 30 percent missing observations are removed. Following this cleaning step, 30 stocks are randomly selected as the research universe. The JKP data used in this thesis therefore consist not of the full market sample, but of a 30-stock subsample obtained after missing-value cleaning.

Second, the Fama-French three factors are obtained from the Kenneth French Data Library [10], one of the most widely used public databases in empirical asset-pricing research. The three factors used here are the market risk premium $R_m - R_f$, the size factor SMB, and the book-to-market factor HML. $R_m - R_f$ measures the excess return on the market portfolio over the risk-free rate. SMB captures the size effect, and HML captures the value effect. These factors are canonical benchmark variables for explaining cross-sectional differences in stock returns.

Third, news-sentiment data are obtained from the Daily News Sentiment Index released by the Federal Reserve Bank of San Francisco [9]. This index is constructed from economically relevant news reports through textual analysis. It uses articles from 24 major U.S. newspapers as the raw sample, retaining only articles of at least 200 words whose topic is identified as economics and whose country

focus is the United States. The article-level sentiment scores are aggregated into a daily time series, with recent news receiving larger weights through a geometrically decaying scheme. The resulting index is well suited as a proxy for high-frequency macroeconomic sentiment.

The study does not employ a massive market-wide cross section but instead focuses on a 30-asset subset. This choice controls experimental complexity and produces a compact portfolio-weight vector that is easier to interpret economically. The final concatenated time series dataset² spans 25 years (1998 Dec 29 - 2023 Dec 29) with $T = 6292$ total time series observations (columns), and a total of $C_y + C_e = 34$ endogenous and exogenous variables (rows) covering multiple market cycles and economic regimes. The resulting combination of a manageable asset universe and a long temporal horizon provides a suitable setting for evaluating the model's portfolio-allocation performance under varying market conditions.

4.2 Model Inputs and Outputs

Let $N = 30$ denote the number of selected stocks (i.e., $N = C_y = 30$), $C_e = 4$ the number of exogenous variables, and $L = 192$ the length of the historical lookback window resulting in about $S = 6000$ "input-output" pairs.

More precisely, the model inputs are constructed using rolling windows with a one-period stride. For the h -th input, let s_h denote the starting time of the input window. Hence, the observations included in this window satisfy $s_h \leq t \leq s_h + L - 1$, $h = 1, \dots, S$. Note that the first window starts at $t = 1$ and the stride is one period, $s_h = h$. For the one-period-ahead allocation problem considered here, the total number of inputs are $S = T - L = 6100$. Consequently, two consecutive input windows overlap in $L - 1$ observations.

The return of stock i at time t is computed as the log return $R_{i,t} = \log(P_{i,t}/P_{i,t-1})$, where $P_{i,t}$ denotes the price of stock i at time t .

The endogenous component of the h -th input is the historical return matrix $X_h^{\text{en}} = [R_{i,t}]_{\substack{1 \leq i \leq N, \\ s_h \leq t \leq s_h + L - 1}} \in \mathbb{R}^{N \times L}$. The corresponding exogenous input is denoted by $X_h^{\text{ex}} \in \mathbb{R}^{C_e \times L}$. The complete model input is obtained by stacking the endogenous and exogenous components: $X_h = [(X_h^{\text{en}})^\top (X_h^{\text{ex}})^\top]^\top \in \mathbb{R}^{(N+C_e) \times L}$.

Given X_h , the portfolio-allocation model introduced in Section 3 produces the next-period portfolio-weight vector $\mathbf{w}_h = f_\theta(X_h) = [w_{1,h}, \dots, w_{N,h}]^\top \in \mathbb{R}^N$. As described in Section 3.5, the model applies a softmax normalization to its output scores. Therefore, by Equation (2), $\mathbf{w}_h$ is a valid long-only, fully invested portfolio-weight vector satisfying $w_{i,h} \geq 0$ for every asset and $\sum_{i=1}^N w_{i,h} = 1$.

The portfolio $\mathbf{w}_h$ is formed using information available through time $s_h + L - 1$ and is held over the immediately following period. The vector of asset returns³ realized during the period $s_h + L$ is

$\mathbf{r}_{s_h+L} = [R_{1,s_h+L}, \dots, R_{N,s_h+L}]^\top$. The corresponding realized portfolio return is $r_{p,s_h+L} = \mathbf{w}_h^\top \mathbf{r}_{s_h+L} = \sum_{i=1}^N w_{i,h} R_{i,s_h+L}$.

Importantly, $\mathbf{w}_h$ is not a precomputed target obtained by solving a separate portfolio-optimization problem for each input. Instead, it is generated directly by the parameterized allocation rule f_θ . The next-period return vector $\mathbf{r}_{s_h+L}$ is not included in the model input and becomes available only after the portfolio has been formed. As we will see in the next section, during training, the realized portfolio returns associated with the inputs in each mini-batch are used to compute the negative Sharpe-ratio loss, which is then backpropagated to estimate the model parameters θ .

During validation and testing, portfolio weights are generated exclusively from the information contained in X_h , while $\mathbf{r}_{s_h+L}$ is used only to evaluate the subsequent out-of-sample portfolio performance. The model therefore learns an end-to-end mapping from historical market information to long-only portfolio allocations without requiring an intermediate prediction of individual stock returns.

4.3 Loss Function

Unlike the conventional two-stage paradigm in financial forecasting, where asset returns are first predicted and then translated into portfolio allocations, this study formulates the problem directly as a portfolio-weight learning task. The optimization objective is aligned with risk-adjusted portfolio performance.

The batch setting is introduced during training. Let $\mathcal{B}_q = \{h_1, \dots, h_B\}$ denote the q -th mini-batch, where B is the batch size. For each input $h \in \mathcal{B}_q$, the model produces weights $\mathbf{w}_h$ and the realized next-period portfolio return is computed as r_{p,s_h+L} .

The sample mean of portfolio returns within the mini-batch is $\bar{r}_{p,\mathcal{B}_q} = \frac{1}{B} \sum_{h \in \mathcal{B}_q} r_{p,s_h+L}$. The corresponding batch variance is $\sigma_{p,\mathcal{B}_q}^2 = \frac{1}{(B-1)} \sum_{h \in \mathcal{B}_q} (r_{p,s_h+L} - \bar{r}_{p,\mathcal{B}_q})^2$. The training loss is the negative

Sharpe ratio computed over the mini-batch: $\mathcal{L}_{\text{train}} = -\frac{\bar{r}_{p,\mathcal{B}_q}}{\sqrt{\sigma_{p,\mathcal{B}_q}^2 + \varepsilon}}$,

where ε is a small numerical stabilization term. Compared with conventional regression losses such as mean squared error (MSE) or mean absolute error (MAE), this objective directly optimizes the trade-off between return and risk. The mini-batch is therefore a training device used to estimate the Sharpe ratio-oriented objective stochastically; it is not part of the model architecture itself.

4.4 Training

We train X-KANformer using the Adam optimizer [21] and the mini-batch negative Sharpe-ratio loss defined above. Selective Learning [11], which is well aligned with financial time series, whose empirical distributions often exhibit volatility clustering, jumps, and outlier events [5] is applied to the training set to automatically mask uncertain and anomalous timesteps, thereby reducing their influence on parameter estimation.

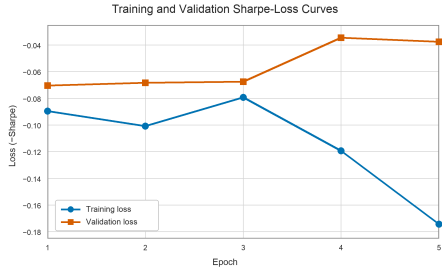
²To identify missing observations during preprocessing, a binary observation mask may be defined as $M_{i,t} = \begin{cases} 1, & \text{if the observation is available,} \\ 0, & \text{if the observation is missing.} \end{cases}$ In the implemented data-loading procedure, the mask is used implicitly to identify missing entries but is not supplied to the model as an additional input. After the stock-level missing-rate filter, the remaining missing stock returns are replaced by zero. Missing external variables arising during date alignment are also replaced by zero before standardisation.

³Intuitively, one may think of $\mathbf{r}_{s_h+L}$ as the corresponding output of the h -th endogenous input X_h^{en} .

Table 1: Ablation results for X-KANformer.

Model variant	KAN	SL	Sharpe	Drop
Full X-KANformer	Yes	Yes	1.375	-
w/o KAN	No	Yes	1.219	11.35%
w/o SL	Yes	No	1.114	18.98%
w/o KAN and SL	No	No	1.031	25.02%

For each mini-batch, the model generates long-only portfolio weights, computes the corresponding next-period portfolio returns, and updates its parameters through backpropagation. At the end of each epoch, performance is evaluated on the validation set, and the checkpoint with the highest validation Sharpe ratio is retained. As shown in Figure 3, the divergence between the training and validation curves after approximately three epochs indicates overfitting and motivates validation-based checkpoint selection. We divide the observations chronologically into training, validation, and test sets using a 7:1:2 ratio, following the training and evaluation protocol as described in Section 2 [38].

**Figure 3: Training and validation Loss(-Sharpe) curves.**

4.5 Results

4.5.1 Ablation Study. To evaluate the individual contributions of the KANs-based feed-forward mapping and SL, we conduct a two-factor ablation study. All four model variants are set the same. Therefore, the differences in out-of-sample performance can be attributed to the inclusion or removal of the components.

In the variant without KAN, the KAN-based nonlinear mapping in each encoder layer is replaced by the conventional Transformer feed-forward network, while the attention architecture and all remaining model components are kept unchanged. In the variant without SL, the complete KAN-enhanced architecture is retained, but all training observations contribute to the loss without selective masking. The final variant removes both KAN and SL, providing a standard Transformer-based portfolio-allocation model as the reference configuration.

As shown in Table 1, the complete X-KANformer configuration achieves the highest out-of-sample Sharpe ratio of 1.375. Replacing the KAN-based mapping with the conventional Transformer feed-forward network reduces the Sharpe ratio to 1.219, representing an 11.35% decline relative to the full model. Since the remaining architecture and training protocol are unchanged, this comparison supports the incremental contribution of KAN-based nonlinear transformations beyond the attention-based backbone.

Removing Selective Learning while retaining the KAN-enhanced architecture reduces the Sharpe ratio to 1.114, corresponding to an 18.98% decline. Under the current experimental setting, the larger reduction associated with removing SL suggests that filtering uncertain and anomalous training observations is particularly important for noisy and heavy-tailed financial time series.

When both KAN and SL are removed, the Sharpe ratio decreases further to 1.031, which is 25.02% below the performance of the complete model. This result suggests that the two components are not redundant. Instead, they appear to improve portfolio generation through complementary channels: KAN strengthens nonlinear feature transformation after attention-based contextualization, whereas SL reduces the influence of less generalizable observations during training. Taken together, the ablation results support the inclusion of both components in the complete X-KANformer architecture.

4.5.2 Leaderboard Comparison. At the time of evaluation, the average Sharpe ratio across all models reported on the leaderboard is 1.321, [CTF Leaderboard], as of August 01, whereas X-KANformer achieves an out-of-sample Sharpe ratio of 1.375. The result indicates that X-KANformer delivers above-average risk-adjusted portfolio performance relative to the broader set of submitted models⁴.

This comparison is particularly informative in light of the relatively restricted information set used in our experiments. X-KANformer is trained on a randomly selected universe of only 30 stocks, together with the Fama–French three factors and a daily news-sentiment index. In contrast, many models on the leaderboard employ substantially larger asset universes, richer sets of stock characteristics and market factors, or more extensive feature-engineering and portfolio-construction procedures. These differences generally provide competing models with access to a broader information set from which predictive signals may be extracted.

Despite operating with a smaller asset universe and a more compact set of input variables, X-KANformer achieves a Sharpe ratio above the leaderboard average. This finding suggests that the proposed combination of attention-based temporal modeling, KANs-based nonlinear transformations, exogenous-information integration, and Selective Learning can extract economically useful allocation signals under comparatively limited data and computational resources. Therefore, although X-KANformer is no longer among the highest-ranked models as the leaderboard has expanded, its performance remains competitive when evaluated relative to the average model and interpreted in the context of its more constrained experimental setting. Moreover, our results were obtained under a relatively modest computational budget compared with many leaderboard submissions, without extensive hyperparameter searches, large-scale model ensembling, or repeated retraining. It should also be emphasized that the primary objective of this study is not to develop a state-of-the-art predictive model optimized for leaderboard performance, but to investigate the usefulness of KANs in financial time-series forecasting. From this perspective, the competitive performance of X-KANformer provides encouraging evidence that KANs-based components can contribute economically

⁴We note X-KANformer ranked in top 10 on the leaderboard at the time of the creation in early May.

meaningful predictive signals even under constrained data and computational settings.

5 Conclusion

This paper investigates the role of KANs as an alternative to the conventional feed-forward neural network within a Transformer architecture for portfolio-oriented financial forecasting. The resulting X-KANformer combines patch-level representations of historical stock returns with variate-level representations of Fama-French factors and market news sentiment. Endogenous self-attention captures dependencies across historical return patches, while cross-attention conditions asset-level representations on the prevailing market state. The KAN module subsequently applies learnable edge-wise nonlinear transformations to the contextualized features. Rather than separately forecasting returns and translating them into allocations through an additional optimization step, the model directly produces long-only, fully invested portfolio weights and is trained using a negative Sharpe-ratio objective. Selective Learning is further employed to reduce the influence of uncertain and anomalous observations during training.

In the empirical analysis, results (Section 4) provide strong evidence that the KAN-based nonlinear mapping and Selective Learning provide complementary improvements: the former strengthens the transformation of attention-based representations, whereas the latter limits the influence of less generalizable financial observations.

These findings should nevertheless be interpreted in light of several limitations. The empirical analysis is based on a randomly selected universe of 30 U.S. stocks, and the leaderboard models do not necessarily use the same information set or portfolio-construction setting. The reported comparison therefore provides useful reference evidence but is not a fully controlled head-to-head evaluation. Moreover, the current analysis does not explicitly incorporate transaction costs, turnover constraints, liquidity restrictions, or other market frictions that may affect implementable portfolio performance. Future research could evaluate the architecture across larger asset universes, alternative markets, and multiple sample periods and examine performance after accounting for trading costs and turnover. Further investigation of the learned spline functions may also clarify which nonlinear market relationships drive the allocation decisions. Overall, the results suggest that combining attention-based temporal modeling, KAN-based nonlinear transformations, and portfolio-aligned training is a promising direction for financial forecasting and allocation research.

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450* (2016).
- [2] Tim Bollerslev. 1986. Generalized autoregressive conditional heteroskedasticity. *Journal of econometrics* 31, 3 (1986), 307–327.
- [3] George Box and GM Jenkins. 1976. Analysis: Forecasting and control. *San francisco* 10 (1976).
- [4] Luyang Chen, Markus Pelger, and Jason Zhu. 2024. Deep learning in asset pricing. *Management Science* 70, 2 (2024), 714–750.
- [5] Rama Cont. 2001. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative finance* 1, 2 (2001), 223.
- [6] Stefan Elfving, Eiji Uchibe, and Kenji Doya. 2018. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks* 107 (2018), 3–11. Special issue on deep reinforcement learning. doi:10.1016/j.neunet.2017.12.012
- [7] Robert F Engle. 1982. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica: Journal of the econometric society* (1982), 987–1007.
- [8] Eugene F Fama and Kenneth R French. 1993. Common risk factors in the returns on stocks and bonds. *Journal of financial economics* 33, 1 (1993), 3–56.
- [9] Federal Reserve Bank of San Francisco. 2026. Daily News Sentiment Index. Accessed: 2026-06-03. <https://www.frbsf.org/research-and-insights/data-and-indicators/daily-news-sentiment-index/>
- [10] Kenneth R. French. 2026. Fama/French Factors. Kenneth R. French Data Library. Accessed: 2026-06-03. https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/Data_Library/f-f_factors.html
- [11] Yisong Fu, Zezhi Shao, Chengqing Yu, Yujie Li, Zhulin An, Qi Wang, Yongjun Xu, and Fei Wang. 2025. Selective Learning for Deep Time Series Forecasting. *arXiv preprint arXiv:2510.25207* (2025).
- [12] Yisong Fu, Zezhi Shao, Chengqing Yu, Yujie Li, Zhulin An, Qi Wang, Yongjun Xu, and Fei Wang. 2026. Selective learning for deep time series forecasting. *Advances in neural information processing systems* 38 (2026), 98084–98115.
- [13] Shihao Gu, Bryan Kelly, and Dacheng Xiu. 2020. Empirical asset pricing via machine learning. *The Review of Financial Studies* 33, 5 (2020), 2223–2273.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [15] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [16] Theis Ingerslev Jensen, Bryan Kelly, and Lasse Heje Pedersen. 2023. Is there a replication crisis in finance? *The Journal of Finance* 78, 5 (2023), 2465–2518.
- [17] Lingyu Jiang, Yuping Wang, Yao Su, Shuo Xing, Wenjing Chen, Xin Zhang, Zhengzhong Tu, Ziming Zhang, Fangzhou Lin, Michael Zielewski, et al. 2025. KANMixer: Can KAN Serve as a New Modeling Core for Long-term Time Series Forecasting? *arXiv preprint arXiv:2508.01575* (2025).
- [18] Bryan Kelly, Semyon Malamud, and Kangying Zhou. 2024. The virtue of complexity in return prediction. *The Journal of Finance* 79, 1 (2024), 459–503.
- [19] Bryan T Kelly and Semyon Malamud. 2025. Understanding the virtue of complexity. *Available at SSRN 5346842* (2025).
- [20] Bryan T Kelly, Semyon Malamud, and Kangying Zhou. 2022. The virtue of complexity everywhere. *Available at SSRN 4166368* (2022).
- [21] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*.
- [22] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. itransformer: Inverted transformers are effective for time series forecasting. In *International conference on learning representations*, Vol. 2024. 11116–11140.
- [23] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. 2022. Non-stationary transformers: Exploring the stationarity in time series forecasting. *Advances in neural information processing systems* 35 (2022), 9881–9893.
- [24] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljacic, Thomas Hou, and Max Tegmark. 2025. KAN: Kolmogorov–Arnold Networks. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. 70367–70413. https://proceedings.iclr.cc/paper_files/paper/2025/file/afaed89642ea100935e39d39a4da602c-Paper-Conference.pdf
- [25] Donghao Luo and Xue Wang. 2024. DeformableTST: Transformer for Time Series Forecasting without Over-reliance on Patching. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=B1lQ1EOiVU>
- [26] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. *arXiv:2211.14730 [cs.LG]* <https://arxiv.org/abs/2211.14730>
- [27] Xihao Piao, Zheng Chen, Taichi Murayama, Yasuko Matsubara, and Yasushi Sakurai. 2024. Fredformer: Frequency debiased transformer for time series forecasting. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*. 2400–2410.
- [28] Tabish Ali Rather, SM Joy, Nadezda Sukhorukova, and Federico Frascoli. 2025. KAN vs LSTM Performance in Time Series Forecasting. *arXiv preprint arXiv:2511.18613* (2025).
- [29] William F Sharpe. 1966. Mutual fund performance. *The Journal of business* 39, 1 (1966), 119–138.
- [30] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research* 15, 1 (2014), 1929–1958.
- [31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. *arXiv:1706.03762 [cs.CL]* <https://arxiv.org/abs/1706.03762>
- [32] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Guo Qin, Haoran Zhang, Yong Liu, Yunzhong Qiu, Jianmin Wang, and Mingsheng Long. 2024. Timexer: Empowering transformers for time series forecasting with exogenous variables. *Advances in Neural Information Processing Systems* 37 (2024), 469–498.

- [33] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2022. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. *arXiv:2106.13008 [cs.LG]* <https://arxiv.org/abs/2106.13008>
- [34] Chao Zhang, Zihao Zhang, Mihai Cucuringu, and Stefan Zohren. 2021. A universal end-to-end approach to portfolio optimization via deep learning. *arXiv preprint arXiv:2111.09170* (2021).
- [35] Zihao Zhang, Stefan Zohren, and Stephen Roberts. 2020. Deep learning for portfolio optimization. *arXiv preprint arXiv:2005.13665* (2020).
- [36] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. *arXiv:2012.07436 [cs.LG]* <https://arxiv.org/abs/2012.07436>
- [37] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting. *arXiv:2201.12740 [cs.LG]* <https://arxiv.org/abs/2201.12740>
- [38] Zhi-Hua Zhou. 2021. *Machine learning*. Springer nature.